

ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 26

GraphQL и Apollo

Рассмотрим парадигму GraphQL как альтернативу REST, систему типов и схем, создание серверной части с Apollo Server, а также подписки (subscriptions) для работы в реальном времени. Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

GraphQL vs REST

При разработке API существуют два основных подхода: **REST** и **GraphQL**. Оба решают задачу организации взаимодействия между клиентом и сервером, однако делают это принципиально по-разному.

REST (Representational State Transfer)

В REST API данные организованы вокруг **ресурсов**: каждый ресурс имеет свой URL, а HTTP-метод (GET, POST, PUT, DELETE) определяет действие над ним.

GET	/users	– получить список пользователей
GET	/users/42	– получить пользователя #42
GET	/users/42/posts	– получить посты пользователя #42
POST	/users	– создать пользователя
DELETE	/users/42	– удалить пользователя #42

Типичные проблемы REST:

- **Overfetching** — сервер возвращает больше данных, чем нужно клиенту. Например, при запросе `/users` клиент получает все поля, хотя ему нужны только имя и аватар.

- **Underfetching** — для построения одного экрана приложения требуется несколько запросов к разным эндпоинтам (N+1 проблема).
- **Версионирование** — при изменении структуры ответа необходимо создавать новые версии API (`/v1/users` , `/v2/users`).

GraphQL

GraphQL — это язык запросов для API и среда выполнения этих запросов на стороне сервера. Разработан в Facebook в 2012 году, опубликован в 2015-м.

Ключевое отличие: клиент **сам описывает**, какие именно поля ему нужны, и всегда получает ровно те данные, которые запросил — не больше и не меньше. При этом у API всего один эндпоинт (обычно `/graphql`).

```
POST /graphql
```

```
{
  "query": "{ user(id: 42) { name avatar posts { title } } }"
}
```

Характеристика	REST	GraphQL
Эндпоинты	Множество (<code>/users</code> , <code>/posts</code> , ...)	Один (<code>/graphql</code>)
Структура ответа	Определяет сервер	Определяет клиент
Overfetching	Частая проблема	Отсутствует
Underfetching	Требуется несколько запросов	Один запрос для любых данных
Версионирование	Нужно (<code>/v1</code> , <code>/v2</code>)	Не требуется (схема расширяется)
Типизация	Опциональная (OpenAPI/Swagger)	Встроенная, строгая
Инструменты	Postman, Swagger UI	GraphQL Playground, Apollo Sandbox

Система типов и схема GraphQL

В основе GraphQL лежит **схема** (Schema) — строгое описание всех доступных данных и операций. Схема пишется на языке **SDL** (Schema Definition Language).

Скалярные типы

```
String    — строка UTF-8
Int       — 32-битное целое число
```

Float — число с плавающей точкой
Boolean — `true` / `false`
ID — уникальный идентификатор (строка или число)

Объектные типы

```
type User {  
  id: ID!  
  name: String!  
  email: String!  
  age: Int  
  posts: [Post!]!  
}  
  
type Post {  
  id: ID!  
  title: String!  
  content: String!  
  author: User!  
  createdAt: String!  
}
```

Восклицательный знак `!` означает, что поле **не может быть null**. `[Post!]!` означает: массив не может быть null, и ни один элемент массива не может быть null.

Корневые типы: Query, Mutation, Subscription

Схема обязательно содержит тип `Query` (операции чтения) и может содержать `Mutation` (операции изменения данных) и `Subscription` (подписки на события).

```
type Query {  
  users: [User!]!  
  user(id: ID!): User  
  posts: [Post!]!  
  post(id: ID!): Post  
}  
  
type Mutation {  
  createUser(name: String!, email: String!): User!  
  updateUser(id: ID!, name: String): User!  
  deleteUser(id: ID!): Boolean!  
  createPost(title: String!, content: String!, authorId: ID!): Post!  
}  
  
type Subscription {  
  postCreated: Post!
```

```
    userUpdated(id: ID!): User!
  }
}
```

Input-типы

Для мутаций с несколькими параметрами удобно использовать `input`-типы:

```
input CreateUserInput {
  name: String!
  email: String!
  age: Int
}

type Mutation {
  createUser(input: CreateUserInput!): User!
}
```

Apollo Server

Apollo Server — наиболее популярная реализация GraphQL-сервера для Node.js. Он принимает схему и резолверы, создаёт HTTP-обработчик и предоставляет встроенный GraphQL Sandbox для тестирования.

Установка

```
npm init -y
npm install @apollo/server graphql
```

Создание сервера

```
// server.js
import { ApolloServer } from '@apollo/server';
import { startStandaloneServer } from '@apollo/server/standalone';

// 1. Определяем схему
const typeDefs = `#graphql
  type User {
    id: ID!
    name: String!
    email: String!
    posts: [Post!]!
  }

  type Post {
    id: ID!
    title: String!
    content: String!
  }
`;
```

```

        author: User!
    }

    type Query {
        users: [User!]!
        user(id: ID!): User
        posts: [Post!]!
    }

    type Mutation {
        createUser(name: String!, email: String!): User!
        createPost(title: String!, content: String!, authorId: ID!): Post!
    }
`;

// 2. Данные в памяти (в реальном проекте – база данных)
const users = [
    { id: '1', name: 'Иван Иванов', email: 'ivan@example.com' },
    { id: '2', name: 'Мария Петрова', email: 'maria@example.com' },
];

const posts = [
    { id: '1', title: 'Первый пост', content: 'Содержимое...', authorId: '1' },
    { id: '2', title: 'Второй пост', content: 'Ещё содержимое...', authorId: '1' },
];

// 3. Определяем резолверы
const resolvers = {
    Query: {
        users: () => users,
        user: (_, { id }) => users.find(u => u.id === id),
        posts: () => posts,
    },
    Mutation: {
        createUser: (_, { name, email }) => {
            const user = { id: String(users.length + 1), name, email };
            users.push(user);
            return user;
        },
        createPost: (_, { title, content, authorId }) => {
            const post = { id: String(posts.length + 1), title, content,
authorId };
            posts.push(post);
            return post;
        },
    },
};

```

```
// Вложенные резолверы для связей между типами
User: {
  posts: (parent) => posts.filter(p => p.authorId === parent.id),
},
Post: {
  author: (parent) => users.find(u => u.id === parent.authorId),
},
};

// 4. Запускаем сервер
const server = new ApolloServer({ typeDefs, resolvers });

const { url } = await startStandaloneServer(server, {
  listen: { port: 4000 },
});

console.log(`GraphQL Server ready at: ${url}`);
```

Резолверы (Resolvers)

Резолвер — это функция, которая возвращает значение для конкретного поля в схеме. Каждое поле в GraphQL потенциально имеет свой резолвер.

Сигнатура резолвера:

```
fieldName: (parent, args, context, info) => { ... }
```

- `parent` — результат выполнения резолвера родительского поля.
- `args` — аргументы, переданные в запросе (например, `id`).
- `context` — объект, общий для всего запроса: обычно содержит информацию о пользователе, подключение к БД.
- `info` — информация о запросе (используется редко).

```
const resolvers = {
  Query: {
    // parent = undefined (корневой резолвер)
    // args = { id: '42' }
    user: (parent, args, context) => {
      // context.db — подключение к базе данных
      return context.db.findUserById(args.id);
    },
  },
  User: {
    // parent = объект User, полученный из резолвера выше
    posts: (parent, args, context) => {
      return context.db.findPostsByAuthorId(parent.id);
    },
  },
};
```

```
    },  
  };  
};
```

Передача контекста (context)

Context используется для передачи общих зависимостей (БД, аутентификация) во все резолверы:

```
const { url } = await startStandaloneServer(server, {  
  listen: { port: 4000 },  
  context: async ({ req }) => {  
    const token = req.headers.authorization || '';  
    const user = verifyToken(token);  
    return { db, currentUser: user };  
  },  
});
```

Примеры запросов в GraphQL

Запросы отправляются через GraphQL Sandbox (доступен по адресу сервера) или через HTTP-клиент.

Query — получение данных

```
# Получить всех пользователей с именем и email  
query {  
  users {  
    id  
    name  
    email  
  }  
}  
  
# Получить конкретного пользователя с его постами  
query GetUserWithPosts($userId: ID!) {  
  user(id: $userId) {  
    name  
    email  
    posts {  
      title  
      content  
    }  
  }  
}
```

Mutation — изменение данных

```
# Создать нового пользователя
mutation {
  createUser(name: "Алексей Сидоров", email: "alex@example.com") {
    id
    name
  }
}

# Создать пост (с переменными)
mutation CreatePost($title: String!, $content: String!, $authorId: ID!) {
  createPost(title: $title, content: $content, authorId: $authorId) {
    id
    title
    author {
      name
    }
  }
}
```

Подписки (Subscriptions) в GraphQL

Subscriptions позволяют клиенту подписаться на события на сервере и получать данные в реальном времени. Вместо HTTP они используют **WebSocket**.

Установка зависимостей для подписок

```
npm install @apollo/server @graphql-subscriptions graphql-ws ws graphql
```

Пример сервера с подписками

```
import { ApolloServer } from '@apollo/server';
import { expressMiddleware } from '@apollo/server/express4';
import { ApolloServerPluginDrainHttpServer } from '@apollo/server/plugin/drainHttpServer';
import { makeExecutableSchema } from '@graphql-tools/schema';
import { WebSocketServer } from 'ws';
import { useServer } from 'graphql-ws/lib/use/ws';
import { PubSub } from 'graphql-subscriptions';
import express from 'express';
import http from 'http';

const pubsub = new PubSub();

const typeDefs = `#graphql
  type Post {
    id: ID!
    title: String!
```



```

    author: String!
  }

  type Query {
    posts: [Post!]!
  }

  type Mutation {
    createPost(title: String!, author: String!): Post!
  }

  type Subscription {
    postCreated: Post!
  }
`;

const resolvers = {
  Query: {
    posts: () => [],
  },
  Mutation: {
    createPost: (_, { title, author }) => {
      const post = { id: String(Date.now()), title, author };
      // Публикуем событие – все подписчики получают этот пост
      pubsub.publish('POST_CREATED', { postCreated: post });
      return post;
    },
  },
  Subscription: {
    postCreated: {
      subscribe: () => pubsub.asyncIterator(['POST_CREATED']),
    },
  },
};

const schema = makeExecutableSchema({ typeDefs, resolvers });

const app = express();
const httpServer = http.createServer(app);

// WebSocket-сервер для подписок
const wsServer = new WebSocketServer({ server: httpServer, path: '/graphql' });
const serverCleanup = useServer({ schema }, wsServer);

const server = new ApolloServer({
  schema,
  plugins: [
    ApolloServerPluginDrainHttpServer({ httpServer }),
    { async serverWillStart() { return { async drainServer() { await

```

```

serverCleanup.dispose(); } } } },
  ],
});

await server.start();
app.use('/graphql', express.json(), expressMiddleware(server));

httpServer.listen(4000, () => {
  console.log('Server ready at http://localhost:4000/graphql');
  console.log('Subscriptions ready at ws://localhost:4000/graphql');
});

```

Пример подписки на клиенте (Apollo Client)

```

import { gql, useSubscription } from '@apollo/client';

const POST_CREATED = gql`
  subscription OnPostCreated {
    postCreated {
      id
      title
      author
    }
  }
`;

function LiveFeed() {
  const { data, loading } = useSubscription(POST_CREATED);

  if (loading) return <p>Ожидание новых постов ... </p>;

  return <p>Новый пост: {data.postCreated.title}</p>;
}

```

GraphQL Playground / Apollo Sandbox

После запуска сервера Apollo автоматически открывает **Apollo Sandbox** по адресу сервера (например, `http://localhost:4000`). Это встроенный IDE для тестирования GraphQL API:

- интерактивная документация, построенная на основе схемы;
- автодополнение при написании запросов;
- история запросов;
- редактор переменных и заголовков HTTP.



Tip

Для быстрой проверки подписок можно использовать онлайн-клиент [Apollo Sandbox](#) — он поддерживает WebSocket-соединения прямо в браузере.

Практическое задание

Необходимо реализовать GraphQL API для управления каталогом книг с использованием Apollo Server.

В рамках выполнения задания требуется:

- определить схему GraphQL с типами `Book` и `Author` (связь «один-ко-многим»: у одного автора много книг);
- реализовать тип `Query` с полями: получение всех книг, получение одной книги по `id`, получение всех авторов;
- реализовать тип `Mutation` для создания книги и автора;
- написать резолверы для всех полей, включая вложенные (поле `author` в типе `Book` и поле `books` в типе `Author`);
- запустить сервер и протестировать минимум три запроса через Apollo Sandbox (скриншоты приложить к отчёту).

Формат отчета

В качестве ответа на данный блок практик студентом подготавливается тематический проект. Критерии в [Практике 28](#)

Литература

1. [Официальная документация GraphQL](#)
2. [Документация Apollo Server](#)
3. [Apollo Client \(React\)](#)